DARSHAN UNIVERSITY

Department of Computer Science & Engineering

Course Code: 2601CS102

Web Designing

B.Tech. First Semester

CHAPTER 5

Tailwind CSS — Utility-First Framework

E-Notes / Study Material (Print-Friendly Edition)

Topic	Coverage
Introduction	What Tailwind is, utility-first philosophy, setup
Colors & Theme Variables	Color palette, opacity, custom theme tokens
Typography	Font family, size, weight, alignment, line-height
Spacing & Sizing	Margin, padding, width, height, gap utilities
Background	Background color, gradients, images, position
Lines	Borders, divide, outline, ring utilities
Layout	Display, position, container, overflow, z-index
Flexbox & Grid	flex/grid utilities, alignment, columns
Effects, Filters, Transforms	Shadows, opacity, blur, rotate, scale
Tables & SVG	Table utilities, fill/stroke for SVG icons
Transition & Animation	Smooth state changes, built-in animations
Interactivity	Hover/focus states, cursor, user-select

Academic Year 2025–26

1. Introduction to Tailwind CSS

1.1 What is Tailwind CSS?

Tailwind CSS is a utility-first CSS framework that provides thousands of small, single-purpose classes — like flex, pt-4, text-center, and bg-blue-500 — which you combine directly in your HTML to build any design without writing custom CSS. It was created by Adam Wathan and released in 2017.

> **Utility-First — What Does That Mean?**
> Instead of writing a custom CSS class like .btn-primary and defining its styles separately,
> you apply many small utility classes directly on the element: bg-blue-500 text-white px-4 py-2 rounded
> Each class does exactly ONE job — background color, text color, padding, or border-radius.

1.2 Tailwind vs Bootstrap — Key Difference

Aspect	Bootstrap	Tailwind CSS
Approach	Component-based (pre-styled .btn, .card)	Utility-first (compose your own design)
Design Freedom	Looks similar unless heavily customized	Every project can look completely unique
Learning Curve	Easier — memorize component names	Steeper — memorize utility classes
File Size	Fixed CSS bundle	Only ships CSS actually used (very small)
Customization	Override via Sass variables	Configure via tailwind.config.js

1.3 Setting Up Tailwind CSS

The fastest way to try Tailwind in a college project is the Play CDN — no build tools or installation required:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My Tailwind Page</title>

  <!-- Tailwind Play CDN — quick setup, great for learning & prototypes -->
  <script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="bg-gray-100">

  <h1 class="text-3xl font-bold text-blue-600 text-center mt-10">
    Hello, Tailwind!
  </h1>

</body>
```

```
</html>
```

> ⚠ **For Real Projects — Use the CLI / npm Install:**
>
> The Play CDN is great for learning but NOT recommended for production (larger file, slower).
>
> Real projects install Tailwind via npm: npm install -D tailwindcss
>
> Then generate a tailwind.config.js file and build an optimized CSS file that only includes the classes you actually used.

2. Colors & Theme Variables

2.1 Tailwind's Color System

Tailwind ships with a large, professionally-designed color palette. Every color has 11 shades, numbered from 50 (lightest) to 950 (darkest), so you always have the right tone available.

Shade	50	100	300	500	700	900	950
Lightness	Lightest	Very light	Light	Base color	Dark	Very dark	Darkest

```
<!-- Text color -->
<p class="text-blue-500">Blue text</p>
<p class="text-red-600">Red text</p>

<!-- Background color -->
<div class="bg-green-100">Light green background</div>
<div class="bg-gray-900 text-white">Dark background</div>

<!-- Border color -->
<div class="border border-purple-400">Purple border</div>

<!-- Color with opacity (modern syntax uses a slash) -->
<div class="bg-blue-500/50">50% opacity blue background</div>
<p class="text-black/70">70% opacity black text</p>
```

Common Color Families	Typical Use
slate, gray, zinc, neutral, stone	Neutral tones for text & backgrounds
red, orange, amber, yellow	Warnings, errors, highlights
green, emerald, teal	Success states, confirmations
blue, sky, indigo	Primary actions, links, brand color
purple, fuchsia, pink, rose	Accents, creative highlights

2.2 Theme Variables (Customizing the Design System)

Tailwind lets you define your own reusable design tokens — colors, fonts, spacing — inside a config file (or directly with CSS variables in newer Tailwind v4). This keeps your entire project visually consistent.

```
// tailwind.config.js
module.exports = {
  theme: {
    extend: {
      colors: {
        brand: {
          light: '#5eead4',
          DEFAULT: '#0d9488',
          dark: '#134e4a',
        },
```

```
      },
      fontFamily: {
        sans: ['Poppins', 'sans-serif'],
      },
    },
  },
}
```

```
<!-- Using your custom "brand" color anywhere in the project -->
<button class="bg-brand hover:bg-brand-dark text-white px-4 py-2 rounded">
  Custom Brand Button
</button>
```

Why Theme Variables Matter:

Define your brand color ONCE in the config file.

Use bg-brand, text-brand, border-brand anywhere in your project.

Need to rebrand later? Change the color in ONE place — it updates everywhere automatically.

3. Typography

3.1 Font Family & Size

```
<p class="font-sans">Sans-serif font (default)</p>
<p class="font-serif">Serif font</p>
<p class="font-mono">Monospace font</p>

<!-- Font sizes — from xs (smallest) to 9xl (largest) -->
<p class="text-xs">Extra small text</p>
<p class="text-base">Base size (16px default)</p>
<p class="text-xl">Extra large text</p>
<p class="text-4xl">Very large heading text</p>
```

Class	Approx. Size
text-xs	12px
text-sm	14px
text-base	16px (default)
text-lg	18px
text-xl / text-2xl	20px / 24px
text-4xl / text-6xl	36px / 60px

3.2 Font Weight, Style & Decoration

```
<p class="font-thin">Thin weight</p>
<p class="font-normal">Normal weight</p>
<p class="font-semibold">Semibold weight</p>
<p class="font-bold">Bold weight</p>
<p class="font-black">Black (heaviest) weight</p>

<p class="italic">Italic text</p>
<p class="underline">Underlined text</p>
<p class="line-through">Strikethrough text</p>
<p class="uppercase">uppercase text</p>
<p class="capitalize">capitalize each word</p>
```

3.3 Text Alignment & Line Height

```
<p class="text-left">Left aligned</p>
<p class="text-center">Center aligned</p>
<p class="text-right">Right aligned</p>
<p class="text-justify">Justified text</p>

<!-- Line height (leading) -->
<p class="leading-tight">Tight line spacing</p>
<p class="leading-normal">Normal line spacing</p>
<p class="leading-loose">Loose line spacing</p>

<!-- Letter spacing (tracking) -->
```

```
<p class="tracking-tight">Tight letters</p>
<p class="tracking-wide">Wide letter spacing</p>

<!-- Truncate long text with ellipsis -->
<p class="truncate w-40">This text will be cut off with an ellipsis...</p>
```

Utility Prefix	Controls
font-{weight}	thin, normal, medium, semibold, bold, black
text-{size}	Font size (xs → 9xl)
text-{align}	left, center, right, justify
leading-{amount}	Line height / spacing between lines
tracking-{amount}	Letter spacing
truncate	Cuts off overflowing text with ellipsis (...)

4. Spacing & Sizing

4.1 The Tailwind Spacing Scale

Tailwind uses a consistent numeric spacing scale. Each step roughly equals 0.25rem (4px), so the numbers scale predictably across your entire project.

Class Number	Value	Class Number	Value
0	0px	4	16px
1	4px	6	24px
2	8px	8	32px
3	12px	12	48px

4.2 Margin & Padding

```
<!-- Padding: p (all sides), px (left+right), py (top+bottom) -->
<div class="p-4">Padding on all sides</div>
<div class="px-6 py-2">Horizontal 6, vertical 2 padding</div>
<div class="pt-8">Padding top only</div>

<!-- Margin: m, mx, my, mt, mb, ml, mr -->
<div class="m-4">Margin on all sides</div>
<div class="mx-auto w-64">Horizontally centered block</div>
<div class="mt-10">Margin top only</div>

<!-- Negative margin -->
<div class="-mt-4">Pulls element up by 4 units</div>
```

Prefix	Side Affected
p- / m-	All four sides
px- / mx-	Left + Right (x-axis)
py- / my-	Top + Bottom (y-axis)
pt-, pb-, pl-, pr-	One specific side (top/bottom/left/right)

4.3 Width, Height & Gap

```
<!-- Width -->
<div class="w-1/2">50% width</div>
<div class="w-full">100% width</div>
<div class="w-64">Fixed width (16rem / 256px)</div>
<div class="w-screen">Full viewport width</div>

<!-- Height -->
<div class="h-32">Fixed height</div>
<div class="h-screen">Full viewport height</div>
<div class="min-h-screen">At least full viewport height</div>
```

```
<!-- Gap — spacing between flex/grid children -->
<div class="flex gap-4">
  <div>Item 1</div>
  <div>Item 2</div>
</div>
```

Quick Tip:

w-1/2, w-1/3, w-2/3, w-1/4 etc. use fraction-based widths — perfect for quick layouts without a full grid. gap-{n} replaces the need for manual margins between flex or grid children.

5. Background

5.1 Background Color & Gradients

```
<!-- Solid background color -->
<div class="bg-teal-600 text-white p-6">Solid background</div>

<!-- Linear gradient -->
<div class="bg-gradient-to-r from-blue-500 to-purple-600 p-6 text-white">
  Left-to-right gradient
</div>

<!-- Gradient with a middle color stop -->
<div class="bg-gradient-to-br from-pink-400 via-red-500 to-yellow-500 p-6">
  Diagonal 3-color gradient
</div>
```

Direction Class	Gradient Flows
bg-gradient-to-r	Left → Right
bg-gradient-to-l	Right → Left
bg-gradient-to-b	Top → Bottom
bg-gradient-to-br	Top-left → Bottom-right (diagonal)

5.2 Background Image, Size & Position

```
<div class="bg-[url('/images/hero.jpg')] bg-cover bg-center bg-no-repeat h-96">
  Hero banner with background image
</div>

<!-- bg-cover  → fills entire element, may crop image     -->
<!-- bg-contain → fits whole image, may leave empty space -->
<!-- bg-center → centers the image within the element     -->
<!-- bg-fixed  → background stays fixed while scrolling    -->
```

Arbitrary Values — [ ] Square Bracket Syntax:

Tailwind lets you use ANY custom value directly in a class using square brackets:

bg-[#1e3a5f] → custom hex color

bg-[url('/img/bg.jpg')] → custom background image path

This gives you full flexibility beyond the built-in design tokens.

6. Lines — Borders, Dividers & Rings

6.1 Border Utilities

```
<!-- Border width & color -->
<div class="border border-gray-300">Default 1px border</div>
<div class="border-2 border-teal-500">2px teal border</div>
<div class="border-t-4 border-red-500">4px border on top only</div>

<!-- Border radius (rounded corners) -->
<div class="rounded">Slightly rounded</div>
<div class="rounded-lg">Large rounded corners</div>
<div class="rounded-full">Fully circular / pill shape</div>
<div class="rounded-t-xl">Rounded top corners only</div>

<!-- Border style -->
<div class="border-dashed border-2">Dashed border</div>
<div class="border-dotted border-2">Dotted border</div>
```

6.2 Divide Utilities — Borders Between Children

The divide-* utilities automatically add a border BETWEEN child elements, without needing a border on the first or last item.

```
<div class="divide-y divide-gray-300">
  <div class="py-2">Item 1</div>
  <div class="py-2">Item 2</div>
  <div class="py-2">Item 3</div>
</div>
<!-- Adds a horizontal line only BETWEEN each item, not around the whole box -->
```

6.3 Ring Utilities — Outline-Style Focus Rings

A 'ring' is a box-shadow-based outline commonly used for accessible focus states on buttons and inputs, without affecting the element's layout/size.

```
<button class="ring-2 ring-blue-500 ring-offset-2 px-4 py-2 rounded">
  Button with a visible ring
</button>

<!-- Common pattern: show ring only when focused -->
<input class="border rounded px-3 py-2 focus:ring-2 focus:ring-teal-500
focus:outline-none">
```

Utility	Purpose
border / border-{n}	Border width
border-{color}	Border color
rounded / rounded-{size}	Border radius
divide-y / divide-x	Borders between children (not around them)

ring-{n} / ring-{color}	Focus-style outline ring

7. Layout

7.1 Display & Position

```
<!-- Display -->
<div class="block">Block-level</div>
<div class="inline-block">Inline-block</div>
<div class="hidden">Hidden (display: none)</div>
<div class="hidden md:block">Hidden on mobile, visible from md upward</div>

<!-- Position -->
<div class="relative">
  <span class="absolute top-0 right-0">Positioned inside parent</span>
</div>
<div class="fixed bottom-4 right-4">Fixed floating button</div>
<nav class="sticky top-0">Sticky navbar</nav>
```

7.2 Container, Overflow & Z-Index

```
<!-- Container — centers content with responsive max-widths -->
<div class="container mx-auto px-4">
  Centered, responsive content
</div>

<!-- Overflow -->
<div class="overflow-hidden">Clips overflowing content</div>
<div class="overflow-auto h-40">Scrollable if content is too tall</div>
<div class="overflow-x-scroll">Always horizontally scrollable</div>

<!-- Z-index (stacking order) -->
<div class="relative z-10">Above</div>
<div class="relative z-0">Below</div>
```

Utility	Purpose
block / inline-block / hidden	Controls how an element displays
static / relative / absolute / fixed / sticky	CSS position property
container	Responsive centered wrapper (like Bootstrap's .container)
overflow-hidden / overflow-auto / overflow-scroll	Controls content overflow behavior
z-{n}	Stack order for overlapping elements

8. Flexbox & Grid

8.1 Flexbox Utilities

```
<!-- Turn on flex -->
<div class="flex">
  <div>Item 1</div>
  <div>Item 2</div>
</div>

<!-- Direction -->
<div class="flex flex-row">Row (default)</div>
<div class="flex flex-col">Column — stacks vertically</div>

<!-- Alignment -->
<div class="flex justify-between items-center h-20">
  <span>Left</span>
  <span>Right</span>
</div>

<!-- Wrap & Grow -->
<div class="flex flex-wrap gap-2">Wraps items onto new lines</div>
<div class="flex-1">Grows to fill available space</div>
```

Class	Purpose
flex	Enables flexbox on the container
flex-row / flex-col	Horizontal or vertical direction
justify-start/center/between/end	Horizontal alignment of children
items-start/center/end	Vertical alignment of children
flex-wrap	Allows items to wrap onto new lines
flex-1 / flex-grow / flex-shrink-0	Controls how an item grows/shrinks

8.2 Grid Utilities

```
<!-- Basic grid with 3 equal columns -->
<div class="grid grid-cols-3 gap-4">
  <div>1</div>
  <div>2</div>
  <div>3</div>
</div>

<!-- Responsive grid — 1 col mobile, 2 tablet, 4 desktop -->
<div class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-4">
  <div class="bg-gray-100 p-4">Card 1</div>
  <div class="bg-gray-100 p-4">Card 2</div>
  <div class="bg-gray-100 p-4">Card 3</div>
  <div class="bg-gray-100 p-4">Card 4</div>
</div>

<!-- Item spans multiple columns -->
<div class="grid grid-cols-4 gap-4">
```

```
  <div class="col-span-2">Spans 2 columns</div>
  <div>Normal</div>
  <div>Normal</div>
</div>
```

Class	Purpose
grid	Enables CSS Grid on the container
grid-cols-{n}	Number of equal-width columns (1–12)
gap-{n}	Spacing between grid cells
col-span-{n}	Makes an item span multiple columns
md:grid-cols-{n}	Different column count per breakpoint

Responsive Prefixes Work on ANY Utility:

sm: (≥640px) md: (≥768px) lg: (≥1024px) xl: (≥1280px) 2xl: (≥1536px)

Example: text-sm md:text-lg lg:text-2xl → font size grows as screen gets wider

Mobile-first: unprefixed classes apply to ALL sizes; prefixed classes override from that breakpoint up

9. Effects, Filters & Transforms

9.1 Effects — Shadows & Opacity

```
<!-- Box shadow -->
<div class="shadow-sm">Small shadow</div>
<div class="shadow-md">Medium shadow</div>
<div class="shadow-xl">Extra large shadow</div>
<div class="shadow-none">No shadow</div>

<!-- Opacity -->
<div class="opacity-100">Fully visible</div>
<div class="opacity-50">50% transparent</div>
<div class="opacity-0">Invisible</div>
```

9.2 Filters

```
<!-- Blur -->
<img class="blur-sm" src="photo.jpg">
<img class="blur-lg" src="photo.jpg">

<!-- Brightness & Contrast -->
<img class="brightness-50" src="photo.jpg">   <!-- darker -->
<img class="brightness-150" src="photo.jpg">  <!-- brighter -->
<img class="contrast-125" src="photo.jpg">

<!-- Grayscale & Saturate -->
<img class="grayscale" src="photo.jpg">
<img class="grayscale-0" src="photo.jpg">  <!-- removes grayscale -->
<img class="saturate-150" src="photo.jpg">

<!-- Backdrop filter — blurs content BEHIND an element -->
<div class="backdrop-blur-md bg-white/30">Frosted glass effect</div>
```

9.3 Transforms

```
<!-- Scale -->
<img class="hover:scale-110 transition" src="card.jpg">

<!-- Rotate -->
<div class="rotate-45">Rotated 45 degrees</div>
<div class="-rotate-12">Rotated -12 degrees</div>

<!-- Translate (move position) -->
<div class="translate-x-4">Moved right</div>
<div class="-translate-y-2">Moved up</div>

<!-- Skew -->
<div class="skew-x-6">Skewed on x-axis</div>

<!-- Combine multiple transforms -->
<div class="hover:scale-105 hover:rotate-1 transition-transform duration-300">
  Card that grows and tilts slightly on hover
</div>
```

Category	Common Classes
Shadows	shadow-sm, shadow-md, shadow-lg, shadow-xl
Opacity	opacity-0 to opacity-100 (in steps of 5/10)
Filters	blur-*, brightness-*, grayscale, saturate-*
Transforms	scale-*, rotate-*, translate-x/y-*, skew-x/y-*

10. Tables & SVG

10.1 Styling Tables with Tailwind

Tailwind has no pre-built 'table component' like Bootstrap — instead, you compose a table's look using ordinary utility classes on <table>, <th>, and <td> elements.

```
<table class="w-full border-collapse">
  <thead class="bg-gray-800 text-white">
    <tr>
      <th class="p-3 text-left">Name</th>
      <th class="p-3 text-left">Branch</th>
      <th class="p-3 text-left">Grade</th>
    </tr>
  </thead>
  <tbody class="divide-y divide-gray-200">
    <tr class="hover:bg-gray-50">
      <td class="p-3">Rahul Patel</td>
      <td class="p-3">CSE</td>
      <td class="p-3">A+</td>
    </tr>
    <tr class="hover:bg-gray-50 bg-gray-50">
      <td class="p-3">Priya Shah</td>
      <td class="p-3">IT</td>
      <td class="p-3">A</td>
    </tr>
  </tbody>
</table>
```

Key Takeaway:

border-collapse merges cell borders into single clean lines.
divide-y on <tbody> creates the row separators (no need for manual per-cell borders).
hover:bg-gray-50 recreates Bootstrap's .table-hover effect using plain utilities.

10.2 Styling SVG Icons

Tailwind provides dedicated utilities to control an inline SVG's fill and stroke colors — perfect for icon libraries like Heroicons or Feather Icons.

```
<!-- Fill color controls the SOLID color of an SVG shape -->
<svg class="w-6 h-6 fill-teal-500" viewBox="0 0 24 24">
  <path d="..."/>
</svg>

<!-- Stroke color controls OUTLINE-style icons -->
<svg class="w-6 h-6 stroke-blue-500" fill="none" stroke-width="2" viewBox="0 0 24
24">
  <path d="..."/>
</svg>

<!-- Size + hover color change together -->
<svg class="w-8 h-8 fill-gray-400 hover:fill-red-500 transition cursor-pointer">
```

```
  <path d="..."/>
</svg>
```

Class	Purpose
fill-{color}	Sets the SVG shape's solid fill color
stroke-{color}	Sets the SVG outline/stroke color
stroke-{width}	Sets the thickness of the stroke line
w-{n} h-{n}	Sets the SVG's rendered width and height

11. Transition & Animation

11.1 Transitions — Smooth Property Changes

The transition-* utilities smoothly animate a property change (like on hover) instead of it snapping instantly.

```
<!-- Basic transition on hover -->
<button class="bg-teal-500 hover:bg-teal-700 transition duration-300 px-4 py-2
rounded text-white">
  Smooth Color Change
</button>

<!-- Transition only specific properties -->
<div class="transition-transform duration-500 hover:scale-110">
  Grows smoothly on hover
</div>

<!-- Timing function (easing) -->
<div class="transition ease-in-out duration-300">Smooth ease-in-out</div>
<div class="transition ease-linear duration-300">Constant linear speed</div>
```

Class	Purpose
transition / transition-all	Enables smooth animation on property changes
transition-colors / transition-transform	Limits animation to specific properties
duration-{ms}	How long the transition takes (150, 300, 500, 700...)
ease-linear / ease-in / ease-out / ease-in-out	Speed curve of the animation
delay-{ms}	Waits before starting the transition

11.2 Built-in Animations

Tailwind includes a few ready-made keyframe animations for common UI patterns:

```
<!-- Spinning loader -->
<div class="animate-spin w-6 h-6 border-4 border-teal-500 border-t-transparent
rounded-full"></div>

<!-- Pulsing skeleton loader -->
<div class="animate-pulse bg-gray-300 h-4 w-full rounded"></div>

<!-- Bouncing icon -->
<span class="animate-bounce">⬇</span>

<!-- Continuous ping (notification dot effect) -->
<span class="relative flex h-3 w-3">
  <span class="animate-ping absolute h-full w-full rounded-full bg-red-400
opacity-75"></span>
  <span class="relative rounded-full h-3 w-3 bg-red-500"></span>
</span>
```

Animation Class	Effect
animate-spin	Continuous 360° rotation — loading spinners
animate-pulse	Fades in and out — skeleton loading placeholders
animate-bounce	Vertical bounce — attention-grabbing icons/arrows
animate-ping	Expanding, fading ring — notification indicators

12. Interactivity

12.1 State Variants — hover, focus, active

Tailwind lets you style DIFFERENT states of an element by simply prefixing any utility class with the state name and a colon.

```
<button class="bg-blue-500 hover:bg-blue-700 active:bg-blue-900
               text-white px-4 py-2 rounded transition">
  Hover & Click Me
</button>

<input class="border border-gray-300 focus:border-teal-500 focus:ring-2
              focus:ring-teal-300 focus:outline-none rounded px-3 py-2">

<!-- group-hover — style a CHILD when the PARENT is hovered -->
<div class="group border p-4 rounded">
  <h3 class="text-gray-800 group-hover:text-teal-600">Card Title</h3>
  <p class="text-gray-400 group-hover:text-gray-700">Description text</p>
</div>
```

State Prefix	Applies When
hover:	Mouse is over the element
focus:	Element is focused (clicked/tabbed into)
active:	Element is actively being clicked
disabled:	Element has the disabled attribute
group-hover:	A PARENT with class="group" is hovered

12.2 Cursor, User-Select & Pointer Events

```
<!-- Cursor style -->
<button class="cursor-pointer">Pointer cursor on hover</button>
<div class="cursor-not-allowed opacity-50">Disabled-looking cursor</div>

<!-- Prevent text selection (useful for buttons/UI, not content) -->
<div class="select-none">Cannot be text-selected</div>

<!-- Disable click/hover interaction entirely -->
<div class="pointer-events-none">Ignores all mouse events</div>
```

12.3 Dark Mode Support

Tailwind has built-in dark mode support using the dark: prefix, which activates automatically based on the user's OS setting (or manually via a toggle class).

```
<body class="bg-white dark:bg-gray-900">
  <h1 class="text-gray-900 dark:text-white">
    This heading adapts to Dark Mode automatically
```

```
  </h1>
  <p class="text-gray-600 dark:text-gray-300">
    Works great for theme switchers in student projects.
  </p>
</body>
```

Key Takeaway:

Almost EVERY Tailwind utility class can be combined with state (hover:, focus:), responsive (md:, lg:), and dark mode (dark:) prefixes.

Example: md:hover:dark:bg-teal-700 → applies only on medium screens, on hover, in dark mode.

Quick Reference — Tailwind CSS Cheat Sheet

Setup

Task	Code
Quick CDN setup	<script src="https://cdn.tailwindcss.com"></script>
Real project install	npm install -D tailwindcss

Colors & Backgrounds

Class	Purpose
text-{color}-{shade}	Text color (e.g. text-blue-500)
bg-{color}-{shade}	Background color
bg-gradient-to-r from-.. to-..	Gradient background
bg-cover / bg-center	Background image sizing/position

Typography

Class	Purpose
text-{size}	Font size (xs → 9xl)
font-{weight}	Font weight (thin → black)
text-{align}	Text alignment
leading-{amount} / tracking-{amount}	Line height / letter spacing

Spacing & Sizing

Class	Purpose
p-{n} / m-{n}	Padding / margin (all sides)
px-, py-, pt-, pb-, pl-, pr-	Padding on specific sides (m- equivalents too)
w-{n} / h-{n}	Width / height
gap-{n}	Spacing between flex/grid children

Layout

Class	Purpose
block / hidden / flex / grid	Display type
relative / absolute / fixed / sticky	Position type

container / overflow-hidden	Layout & content clipping
z-{n}	Stack order

Flexbox & Grid

Class	Purpose
flex-row / flex-col	Flex direction
justify-{value} / items-{value}	Alignment of flex/grid children
grid-cols-{n} / col-span-{n}	Grid columns & spanning

Effects, Filters & Transforms

Class	Purpose
shadow-{size} / opacity-{n}	Box shadow depth / transparency
blur-{size} / grayscale / brightness-{n}	Image/element filters
scale-{n} / rotate-{n} / translate-x/y-{n}	Transform utilities

Transition & Animation

Class	Purpose
transition / duration-{ms} / ease-*	Smooth property change on state change
animate-spin / pulse / bounce / ping	Built-in keyframe animations

Interactivity

Class	Purpose
hover: / focus: / active: / disabled:	State-based variants (prefix any class)
group / group-hover:	Style a child when its parent is hovered
dark:	Dark mode variant (prefix any class)
cursor-pointer / select-none	Cursor & text-selection behavior

Practice Questions — Tailwind CSS

Instructions:
Attempt all questions. Short answer questions carry 2 marks each. Long answer / coding questions carry 5 marks each.

Section A — Short Answer Questions (2 marks each)

1. What is Tailwind CSS? Explain the meaning of "utility-first" framework.
2. Differentiate between Tailwind CSS and Bootstrap in terms of design approach.
3. What is the purpose of theme variables / tailwind.config.js?
4. What does the class bg-blue-500/50 mean?
5. What is the difference between p-4 and px-4 in Tailwind?
6. What is the purpose of the gap-{n} utility?
7. Explain the difference between bg-cover and bg-contain.
8. What does the divide-y utility do, and how is it different from a normal border?
9. What is the purpose of the group and group-hover: utilities?
10. How does Tailwind implement dark mode using the dark: prefix?

Section B — Long Answer Questions (5 marks each)

1. Explain Tailwind's responsive design system using breakpoint prefixes (sm, md, lg, xl, 2xl) with a code example showing a 3-column responsive grid. (5 marks)
2. Explain Flexbox utilities in Tailwind. Build a navbar layout with a logo on the left and navigation links on the right using flex utilities. (5 marks)
3. Explain the difference between Transitions and Animations in Tailwind. List and explain all four built-in animation classes with their use cases. (5 marks)
4. Explain state variants in Tailwind (hover:, focus:, active:, disabled:). Create a styled button and form input demonstrating each state. (5 marks)
5. Explain the CSS Grid utilities in Tailwind including grid-cols-*, gap-*, and col-span-*. Build a 4-column responsive product card grid. (5 marks)
6. Explain filters and transforms in Tailwind CSS. Create a product image card that scales up and adds a shadow on hover using a smooth transition. (5 marks)

Section C — Coding Exercises

1. Create a pricing card component with a heading, price, feature list, and a call-to-action button using Tailwind spacing, typography, and shadow utilities. (5 marks)
2. Build a styled HTML table showing 5 students with columns Roll No, Name, Branch, CGPA — using border-collapse, divide-y, and hover: row highlighting. (4 marks)

3. Create a responsive navigation bar that is a horizontal flex row on desktop and stacks vertically on mobile using the flex-col / md:flex-row utilities. (5 marks)

4. Design a notification bell icon using an inline SVG styled with fill-{color}, and add an animate-ping badge to represent unread notifications. (3 marks)